Command Line Survival Guide

A Practical Reference for MATH 5200

Table of contents

1 Command Line in One Minute **2**

2 Where Am I? **3**
2.1 . and 4

3 Listing Files **5**

4 Moving Around **5**

5 Relative and Absolute Paths **6**
5.1 Why this matters in Python 7

6 Creating and Managing Directories and Files **7**
6.1 `mkdir` 7
6.2 `cp` 8
6.3 `mv` 8
6.4 `rm` 8

7 The File System as a Tree **8**

8 Spaces and Quoting **9**

9 Programs Are Commands Too **10**

10 PATH: How Does the Shell Find a Program? **11**
10.1 Inspecting PATH 11
10.2 Finding an executable 11

11 Environment Variables **12**

12 Running Python from the Command Line **13**

13 `python -m` **13**
13.1 Why `python -m pip`? 14

14 Virtual Environments **14**
14.1 Activating the environment 14
14.2 Why use virtual environments? 15

15 Do You Have the Right Python? **16**

16 Git Lives in the Same World **16**

17 Shell vs. Python vs. Git **17**

18 Commands, Options, and Arguments **18**

19 Getting Help **18**

20 Common Problems and Diagnosis **19**
20.1 "Command not found" 19
20.2 "File not found" 20
20.3 "Python cannot import my package" 20

21 Minimal Command-Line Reference **21**
21.1 Files and directories 21
21.2 Python 22
21.3 Git 23

22 Short Practice Exercise **24**

23 The Mental Model to Remember **25**
23.0.1 1. The shell has a current directory 25
23.0.2 2. Paths can be relative or absolute 25
23.0.3 3. The shell launches programs 26
23.0.4 4. Python and the shell are different languages 26
23.0.5 5. Your computational environment matters 26

24 Command Line, Git, and Reproducible Computing **26**

1 Command Line in One Minute

The **command line** is a way to interact with your computer by typing commands rather than clicking through graphical interfaces.

Three ideas are enough to get started:

1. A **terminal** is the application in which you type commands.
2. A **shell** is the program that reads those commands and executes them.
3. The shell always has a **current working directory**: the folder in which it is currently operating.

A typical workflow looks like this:

```
computer / operating system
          |
        shell
          |
    +-----+-----+---------+
    |           |         |
  Python      Git       Quarto
    |
 Python program
```

The shell is not Python. When you type

```
python
```

the shell starts Python.

When you type

```
git status
```

the shell starts Git.

When you type

```
print("hello")
```

you are writing Python code, not a shell command.

2 Where Am I?

Your shell always has a **current working directory**.

On macOS and Linux:

```
pwd
```

On Windows PowerShell:

```
Get-Location
```

PowerShell also accepts `pwd` as an alias.

The current directory matters because many commands interpret filenames and paths relative to it.

If your current directory is

```
C:\Users\Alex\MATH_5200
```

then

```
data\returns.csv
```

refers to the file

```
C:\Users\Alex\MATH_5200\data\returns.csv
```

The same idea on macOS/Linux might look like:

```
/Users/alex/MATH_5200/data/returns.csv
```

2.1 . and ..

Two special path names are worth knowing:

- . means **the current directory**
- .. means **the parent directory**

For example:

```
cd ..
```

moves up one level.

You can also refer explicitly to the current directory:

```
ls .
```

or:

```
python ./analysis.py
```

3 Listing Files

The standard command on macOS/Linux is:

```
ls
```

It lists the contents of the current directory.

In Windows PowerShell, the native command is:

```
Get-ChildItem
```

but `ls` is also available as a PowerShell alias.

You can list another directory without moving into it:

```
ls data
```

The exact options vary by shell. The important habit is:

> **List the contents of a directory before you start guessing what is there.**

4 Moving Around

The basic command for changing directories is:

```
cd
```

For example:

```
cd data
```

moves into a directory named `data`.

Move back to the parent directory:

```
cd ..
```

You can move through several levels at once:

```
cd projects/MATH_5200/data
```

A path such as

```
projects/MATH_5200/data
```

is a **relative path**: it is interpreted starting from the current directory.

5 Relative and Absolute Paths

A **relative path** tells the computer where something is relative to the current directory.

An **absolute path** specifies the complete location.

Windows:

```
C:\Users\Alex\MATH_5200\data\returns.csv
```

macOS/Linux:

```
/Users/alex/MATH_5200/data/returns.csv
```

A relative path might instead be:

```
data/returns.csv
```

These can refer to the same file if the current directory is the `MATH_5200` directory.

5.1 Why this matters in Python

Suppose your Python program contains:

```
import pandas as pd

df = pd.read_csv("data/returns.csv")
```

The path is interpreted relative to the **Python process's current working directory**.

You can inspect that directory from Python:

```
from pathlib import Path

Path.cwd()
```

This is one of the first things to check when Python says it cannot find a file.

> A file's location and your program's current working directory are not the same thing.

6 Creating and Managing Directories and Files

These commands are enough for most routine coursework.

Task	macOS/Linux	Windows PowerShell
Show current directory	`pwd`	`Get-Location` / `pwd`
List files	`ls`	`Get-ChildItem` / `ls`
Change directory	`cd`	`cd`
Create directory	`mkdir`	`mkdir`
Copy	`cp`	`Copy-Item` / `cp`
Move/rename	`mv`	`Move-Item` / `mv`
Remove	`rm`	`Remove-Item` / `rm`

6.1 `mkdir`

Create a directory:

```
mkdir data
```

Then:

```
cd data
```

6.2 `cp`

Copy a file:

```
cp original.csv backup.csv
```

6.3 `mv`

Move or rename a file:

```
mv old_name.csv new_name.csv
```

Renaming is conceptually just a move within the same directory.

6.4 `rm`

Remove a file:

```
rm old_name.csv
```

Be careful with `rm` and recursive deletion. Command-line deletion generally bypasses the graphical recycle-bin/trash workflow. Do not use recursive deletion commands unless you understand exactly what they will remove.

7 The File System as a Tree

It helps to imagine your computer's file system as a tree:

```
MATH_5200/
   notes/
       python_objects.qmd
       command_line.qmd
   data/
       prices.csv
       returns.csv
   src/
       analysis.py
   README.md
```

If your current directory is `MATH_5200/`, then:

```
data/returns.csv
```

means "go into `data`, then find `returns.csv`."

If your current directory is `MATH_5200/data/`, then:

```
../README.md
```

means "go up one level, then find `README.md`."

Thinking in terms of this tree makes `cd`, relative paths, Git repositories, and Python file paths much easier to understand.

8 Spaces and Quoting

Spaces normally separate arguments to a shell command.

Suppose a directory is named:

```
My Project
```

This may not work as intended:

```
cd My Project
```

Quote the path instead:

```
cd "My Project"
```

As a practical rule:

> If a path contains spaces, quote it.

9 Programs Are Commands Too

The command line is also a way to **launch programs**.

For example:

```
python
```

starts Python.

```
git
```

starts Git.

```
quarto
```

starts Quarto.

And:

```
python analysis.py
```

asks Python to execute `analysis.py`.

The shell therefore acts as a coordinator:

```
shell
 |
 +-- create directories
 +-- move files
 +-- start Python
 +-- start Git
 +-- start Quarto
 +-- run other tools
```

10 PATH: How Does the Shell Find a Program?

When you type:

```
python
```

you usually do not provide the full path to the Python executable.

One important mechanism is the **PATH environment variable**.

`PATH` is essentially a list of directories that the operating system searches for executable programs.

Conceptually:

```
PATH =
    directory A
    directory B
    directory C
    ...
```

When you type `python`, the shell searches those locations until it finds an appropriate executable.

10.1 Inspecting PATH

macOS/Linux:

```
echo $PATH
```

PowerShell:

```
$env:PATH
```

10.2 Finding an executable

Windows:

```
where.exe python
```

macOS/Linux:

```
which python
```

These commands can help diagnose situations in which the wrong Python installation is being used.

11 Environment Variables

An **environment variable** is a named piece of information made available to processes launched by the operating system or shell.

Common examples include:

```
PATH
HOME
USER
```

The exact set differs across operating systems.

Programs launched by the shell can generally read the environment. Python can inspect it:

```
import os

os.environ["PATH"]
```

You can also inspect the entire environment:

```
os.environ
```

The relationship is:

```
shell environment
       |
       | launches
       v
     Python
       |
       | inherits environment
       v
 Python program
```

Environment variables are useful for configuration and other information that should not necessarily be hard-coded into a program.

12 Running Python from the Command Line

The simplest command is:

```
python
```

This starts the Python interpreter interactively.

You can also run a Python file:

```
python analysis.py
```

Or execute a short expression:

```
python -c "print(2 + 3)"
```

You may encounter:

```
python3
```

on some macOS/Linux systems. Whether `python` or `python3` is appropriate depends on how Python is installed.

13 `python -m`

The `-m` option tells Python to run a **module as a program**.

Two especially important examples are:

```
python -m pip install numpy
```

and:

```
python -m venv .venv
```

This is different from:

```
python some_script.py
```

The latter executes a file; `python -m something` asks the Python interpreter to locate and execute a module named `something`.

13.1 Why `python -m pip`?

You will often see:

```
pip install numpy
```

But:

```
python -m pip install numpy
```

explicitly asks **this Python interpreter** to run its associated `pip`.

That helps avoid the common problem:

```
"pip installed the package,
but Python cannot import it."
```

Often the underlying issue is that `pip` and `python` refer to different installations or environments.

A good default habit is:

```
python -m pip install package_name
```

14 Virtual Environments

A **virtual environment** gives a project its own isolated Python environment and installed packages.

Create one with:

```
python -m venv .venv
```

This creates a directory named `.venv`.

14.1 Activating the environment

Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

macOS/Linux:

```
source .venv/bin/activate
```

After activation, commands such as:

```
python
```

and:

```
python -m pip
```

should refer to the Python installation in that virtual environment.

The shell prompt often changes to indicate that the environment is active.

14.2 Why use virtual environments?

Suppose Project A requires one version of a package while Project B requires another.

With virtual environments:

```
Project A
   |
   +-- .venv
       +-- Python
       +-- packages

Project B
   |
   +-- .venv
       +-- Python
       +-- packages
```

Each project can maintain its own computational environment.

Virtual environments are therefore part of **reproducible computational practice**, not merely a Python trick.

15 Do You Have the Right Python?

When something goes wrong, do not guess. Ask Python.

Check the version:

```
python --version
```

Find the exact Python executable:

```
python -c "import sys; print(sys.executable)"
```

Find the `pip` associated with that interpreter:

```
python -m pip --version
```

These answer:

1. What Python version am I running?
2. Which Python installation am I running?
3. Which `pip` belongs to that Python?

16 Git Lives in the Same World

Git is another command-line program.

Check that it is installed:

```
git --version
```

Some Git commands you will use in MATH 5200 include:

```
git status
git init
git clone
git add
git commit
git pull
git push
```

For example:

```
git status
```

means:

1. the shell reads `git status`;
2. the shell finds the Git executable;
3. Git starts;
4. Git interprets `status` as its subcommand;
5. Git reports the state of the repository.

The same general pattern applies to Python and Quarto.

17 Shell vs. Python vs. Git

What you type	Interpreted by	Meaning
`cd data`	Shell	Change the shell's current directory
`ls`	Shell	List directory contents
`python analysis.py`	Shell → Python	Start Python and run a file
`print(x)`	Python	Execute Python code
`import numpy as np`	Python	Import a Python package
`git status`	Shell → Git	Start Git and run `status`
`quarto render`	Shell → Quarto	Start Quarto and render a document

For example, this is **not** Python:

```
cd data
```

And this is **not** a shell command:

```
x = [1, 2, 3]
```

Keeping the two worlds separate prevents many beginner errors.

18 Commands, Options, and Arguments

A command often has a structure like:

```
program   options   arguments
```

For example:

```
python -m pip install numpy
```

can be read conceptually as:

```
python       program
-m           option
pip          module
install      pip subcommand
numpy        argument
```

Another example:

```
git commit -m "Initial analysis"
```

contains:

```
git                  program
commit               subcommand
-m                   option
"Initial analysis"   argument
```

Different programs use different conventions. The important idea is:

> A command is an instruction to a program, followed by information that tells the program what to do.

19 Getting Help

Many command-line programs provide help:

```
python --help
```

```
git --help
```

and often:

```
some_command --help
```

PowerShell also has:

```
Get-Help
```

Python has its own help system once you are inside Python:

```
help()
```

or:

```
help(str)
```

The shell and Python therefore have different documentation and help systems.

20 Common Problems and Diagnosis

20.1 "Command not found"

Possible causes include:

- the program is not installed;
- the executable is not on PATH;
- a virtual environment is not activated;
- you are using a different shell/environment than expected.

For Python:

```
python --version
```

```
python -c "import sys; print(sys.executable)"
```

For Git:

```
git --version
```

20.2 “File not found”

First ask:

> Where am I?

In the shell:

```
pwd
```

or in Python:

```
from pathlib import Path

Path.cwd()
```

Then ask:

> What files are actually here?

Use:

```
ls
```

or the appropriate PowerShell command.

Do not immediately change the Python code. First verify your assumptions about the file system.

20.3 “Python cannot import my package”

Check:

```
python -m pip --version
```

and:

```
python -c "import sys; print(sys.executable)"
```

Then verify that the environment you intend to use is active.

A package installed into one Python environment is not automatically installed into every other environment.

21 Minimal Command-Line Reference

21.1 Files and directories

```
pwd
```

Where am I?

```
ls
```

What is here?

```
cd DIRECTORY
```

Move into a directory.

```
cd ..
```

Move to the parent directory.

```
mkdir DIRECTORY
```

Create a directory.

```
cp SOURCE DESTINATION
```

Copy something.

```
mv SOURCE DESTINATION
```

Move or rename something.

```
rm FILE
```

Remove a file.

On Windows PowerShell, the native names are `Get-Location`, `Get-ChildItem`, `Copy-Item`, `Move-Item`, and `Remove-Item`, although common aliases such as `pwd`, `ls`, `cp`, `mv`, and `rm` are available.

21.2 Python

```
python
```

Start Python interactively.

```
python SCRIPT.py
```

Run a Python program.

```
python -m MODULE
```

Run a Python module.

```
python -m venv .venv
```

Create a virtual environment.

```
python -m pip install PACKAGE
```

Install a package using that Python interpreter's `pip`.

```
python --version
```

Check the Python version.

```
python -c "import sys; print(sys.executable)"
```

Find the Python executable.

21.3 Git

```
git --version
```

Check the Git version.

```
git status
```

Check repository status.

```
git init
```

Create a Git repository.

```
git clone URL
```

Copy an existing repository.

```
git add FILE
```

Stage a file.

```
git commit
```

Create a commit.

```
git pull
```

Retrieve and integrate changes.

```
git push
```

Send commits to a remote repository.

22 Short Practice Exercise

Open a terminal and work through this sequence.

First:

```
pwd
```

Then:

```
ls
```

Create a practice directory:

```
mkdir command_line_practice
```

Move into it:

```
cd command_line_practice
```

Confirm your location:

```
pwd
```

Create a Python virtual environment:

```
python -m venv .venv
```

Activate it using the instructions for your operating system.

Then check:

```
python --version
```

and:

```
python -c "import sys; print(sys.executable)"
```

Finally:

```
ls
```

You should now be able to see the `.venv` directory.

The point is not to memorize commands. Practice the sequence:

```
navigate
   ↓
create
   ↓
enter
   ↓
run a program
   ↓
inspect the result
```

23 The Mental Model to Remember

If you remember only five things:

23.0.1 1. The shell has a current directory

```
pwd
cd
ls
```

help you understand and change where you are.

23.0.2 2. Paths can be relative or absolute

```
data/returns.csv
```

is relative.

```
C:\Users\Alex\MATH_5200\data\returns.csv
```

is absolute on Windows.

23.0.3 3. The shell launches programs

```
python
git
quarto
```

are programs that the shell can start.

23.0.4 4. Python and the shell are different languages

```
cd data
```

is a shell command.

```
import pandas as pd
```

is Python.

23.0.5 5. Your computational environment matters

These commands tell you what Python you are actually using:

```
python --version
python -c "import sys; print(sys.executable)"
python -m pip --version
```

Understanding this is essential for virtual environments, package management, Git-based projects, and reproducible computational work.

24 Command Line, Git, and Reproducible Computing

The command line may initially seem like an alternative to clicking on folders and applications. In computational work, however, it provides something more important:

> **A precise, reproducible way to tell your computer what to do.**

Consider a typical MATH 5200 project:

```
terminal
   |
   +-- cd project
   |
   +-- activate Python environment
   |
   +-- git pull
   |
   +-- python src/analyze.py
   |
   +-- quarto render report.qmd
   |
   +-- git add ...
   |
   +-- git commit
   |
   +-- git push
```

Each step can be written down, repeated, inspected, and eventually automated.

That is the deeper reason to learn the command line: it is a foundation for moving from **manually operating a computer** toward **building computational workflows**.